

UNIT-5

Introduction to Transaction Processing

Transaction:

- ✓ Transaction is a set of operations which are all logically related.
- or
- ✓ Transaction is a single logical unit of work formed by a set of operations

Example: Bank Transaction

Suppose a bank employee transfers ₹800 from account X to account Y.

This transaction consists of the following operations:

Transaction T:

1. Read(X)
2. $X = X - 800$
3. Write(X)
4. Read(Y)
5. $Y = Y + 800$
6. Write(Y)

Operations in Transaction:

The main operations in a transaction are

1. Read Operation
2. Write Operation

1. Read Operation:

Read operation reads the data from the database and then stores it in the buffer in main memory.

Example:

Read(X)

This reads value of X from the database and will store it in the buffer in main memory.

2. Write Operation:

Write operation writes the updated data value back to the database from the buffer.

Example:

Write(X)

This writes the updated value of A from the buffer to the database.

Let's take an example to debit transaction from an account which consists of following operations:

1. R(X);
2. $X = X - 500$;
3. W(X);

Let's assume the value of X before starting of the transaction is 4000.

- ✓ The first operation reads X's value from database and stores it in a buffer.
- ✓ The second operation will decrease the value of X by 500. So buffer will contain 3500.
- ✓ The third operation will write the buffer's value to the database. So X's final value will be 3500.

But it may be possible that because of the failure of hardware, software or power, etc. that transaction may fail before finished all the operations in the set.

For example: If the transaction fails after executing operation 2, the database retains the old value (4000). This indicates that the transaction is incomplete.

To solve this problem, we have two important operations:

- ✓ **Commit:** Commit is the operation that saves the changes made by a transaction permanently in the database.
- ✓ **Rollback:** Rollback is the operation that cancels the changes made by a transaction and returns the database to its previous state.

Database Buffer

Database Buffer in DBMS: A **database buffer** is a temporary storage area in main memory used by DBMS to store recently used data. It helps to avoid accessing the disk again and again, which improves performance.

Buffer Management in DBMS: Buffer management is the process of managing the database buffer.

Its main goal is to:

- ✓ Use memory efficiently
- ✓ Reduce disk I/O operations

The following are some of the key aspects of buffer management:

1. **Buffer Pool:** The buffer pool is the portion of memory allocated to the database buffer. It is divided into fixed-sized blocks, and each block can store a page of data from the database.
2. **Buffer Replacement:** When the buffer pool is full, the DBMS replaces some data in memory with new data.

There are several algorithms used for replacement, such as:

- ✓ Least Recently Used (LRU)
- ✓ Clock
- ✓ Random Replacement

These algorithms decide which pages should be removed from memory.

3. **Buffer Prefetching:** Buffer prefetching means loading data in advance based on future needs. This helps to reduce disk I/O operations and improves system performance.
4. **Dirty Page Management:** When data is modified in memory, it becomes a **dirty page**. These pages must be written back to disk. Dirty page management ensures that modified pages are saved properly and efficiently.
5. **Tuning:** Buffer management requires tuning to use memory efficiently.

This includes:

- ✓ Setting buffer pool size
- ✓ Choosing replacement algorithms
- ✓ Adjusting prefetching based on usage

Types of Database Buffer in DBMS: There are two main types of database buffer in a DBMS:

1. **Buffer Pool:** The buffer pool is the primary database buffer in a DBMS. It is a portion of memory allocated to hold pages of data from the database. The buffer pool is divided into fixed-sized blocks, each of which can store a page of data.

When a page is requested by a user or application:

- ✓ The DBMS checks if it is already in the buffer pool

- ✓ If yes → it is retrieved quickly from memory
- ✓ If no → it is loaded from disk (takes more time)

2. Cache Buffer: In addition to the buffer pool, some DBMSs use a cache buffer to improve performance.

- ✓ The cache buffer is a portion of memory used to store frequently accessed data.
- ✓ The buffer pool stores **database pages**
- ✓ The cache buffer stores **results of recent queries**

When a query is executed:

- ✓ Results are stored in the cache buffer
- ✓ If the same query runs again → results are retrieved quickly from memory
- ✓ No need to execute the query again

Advantages of Database Buffer in DBMS:

- ✓ **Improved Performance:** Stores frequently used data in memory → faster access than disk
- ✓ **Reduced Disk Access:** Decreases disk I/O operations → improves efficiency and disk life
- ✓ **Efficient Use of Memory:** Uses memory effectively by storing frequently accessed data
- ✓ **Reduced Network Traffic:** Data is available in memory → less need to fetch over network
- ✓ **Increased Concurrency:** Multiple users can access data quickly without delays

Disadvantages of Database Buffer in DBMS

- ✓ **Cost:** Requires additional memory → expensive for large systems
- ✓ **Limited Capacity:** Buffer size is limited → needs replacement of data
- ✓ **Data Consistency Issues:** Improper handling may lead to data loss or inconsistency
- ✓ **Increased Complexity:** Makes DBMS more complex to manage and maintain
- ✓ **Latency Issues:** Poor configuration → frequent cache misses → slow performance

Type of Failures in Transaction

Failure Classification: To identify where the problem has occurred, failures are classified into:

- ✓ **Transaction Failure**
- ✓ **System Crash**
- ✓ **Disk Failure**

Transaction failure: Transaction failure occurs when a transaction cannot complete its execution.

Reasons:

- ✓ **Logical Errors:** Due to code or internal errors
- ✓ **System Errors:** DBMS aborts the transaction (e.g., deadlock, resource unavailability)

System Crash: System crash occurs due to failure of hardware or software.

Examples:

- ✓ Power failure
- ✓ Operating system error

Note:

- ✓ Data in main memory is lost

- ✓ Data in disk (non-volatile storage) remains safe

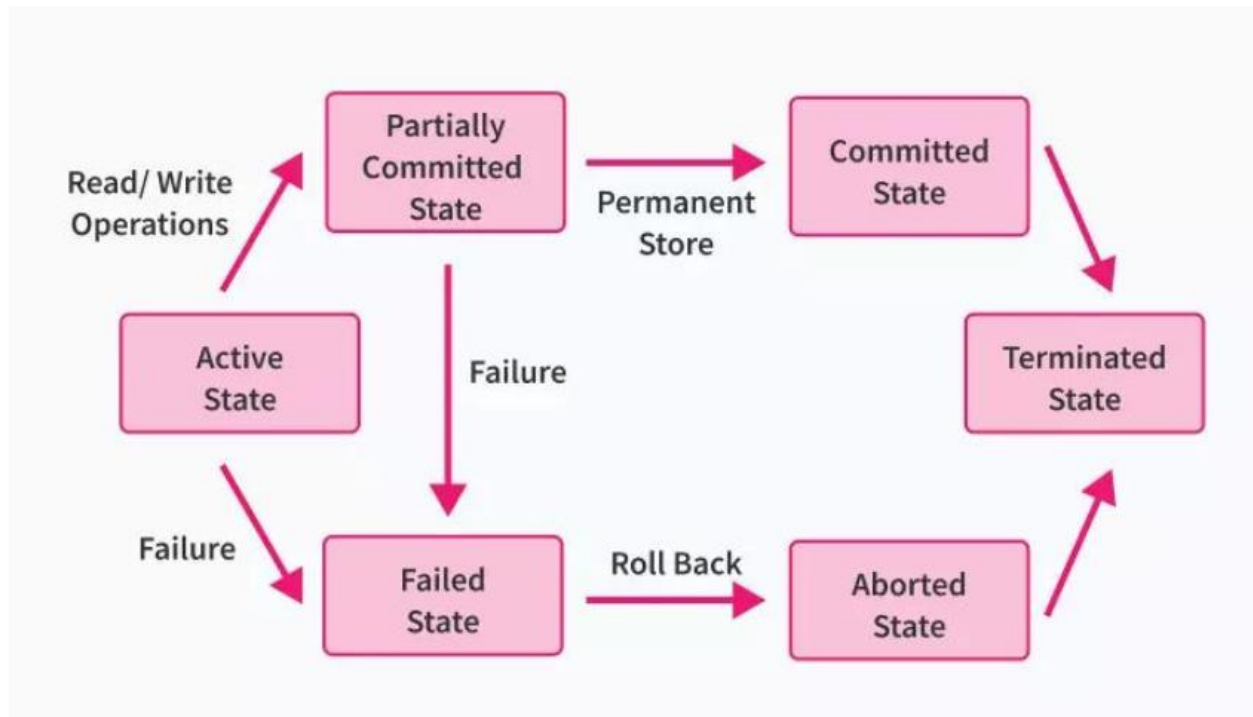
Disk Failure: Disk failure occurs when the storage device is damaged or inaccessible.

Reasons:

- ✓ Bad sectors
- ✓ Disk head crash
- ✓ Disk not reachable

It may destroy some or all data on disk

States of Transaction



States of Transaction

A transaction passes through the following states during its execution:

Example: Bank Transaction

Transfer ₹500 from account X to Y

Transaction T:

Read(X)
 $X = X - 500$
Write(X)
Read(Y)
 $Y = Y + 500$
Write(Y)
Commit

1. Active State:

The transaction is in execution. Read and Write operations are performed

Example:

Read(X), $X = X - 500$, Write(X), Read(Y), $Y = Y + 500$

2. Partially Committed State:

All operations are completed. Commit is not yet executed

Example:

After Write(Y), before Commit

3. Committed State:

Transaction is completed successfully. Changes are permanently saved in the database

Example:

After Commit → X and Y values updated permanently

4. Failed State:

Transaction cannot continue due to error

Example:

Failure occurs after Write(X) (before completing all steps)

5. Aborted State:

Transaction is rolled back. Database is restored to previous consistent state

Example:

Rollback → X returns to original value, Y unchanged

After Aborted

- ✓ Restart the transaction or terminate (kill) the transaction

System Log

Log-Based Recovery:

- ✓ A log is a sequence of records stored in stable storage.
- ✓ It is used to recover the database in case of failure.
- ✓ Every operation performed on the database is recorded in the log.
- ✓ Logs are written **before the actual database update** (Write-Ahead Logging).

Example

Transaction Tn modifies City from *Guntur* → *Hyderabad*

- ✓ When transaction starts:
<Tn, Start>
- ✓ When value is updated:
<Tn, City, 'Guntur', 'Hyderabad'>
- ✓ When transaction completes:
<Tn, Commit>

Recovery Using Log Records

After a system crash, the log is used to decide:

Redo (Re-execute)

- ✓ If log contains <Ti, Start> and <Ti, Commit>, then the Transaction Ti needs to be redone.

Undo (Rollback)

- ✓ If log contains <Ti, Start> but no <Ti, Commit> or <Ti, Abort>, then the Transaction Ti needs to be undone.

Transaction Properties

Transaction Properties (ACID Properties):

A transaction has four

important properties called **ACID properties**.

These properties ensure **consistency of the database** before and after a transaction.

1. Atomicity
2. Consistency
3. Isolation
4. Durability

1. Atomicity

- ✓ It means **all operations of a transaction are executed completely or not at all.**
- ✓ No partial execution is allowed.

Operations:

- ✓ **Commit:** Changes are saved permanently
- ✓ **Abort:** Changes are undone

Example: Consider a transaction T with T1 and T2. Account A has Rs 600 and account B has Rs 300. The transaction transfers Rs 100 from A to B.

T1	T2
Read(A)	Read(B)
A:= A-100	B:= B+100
Write(A)	Write(B)

- ✓ After transaction $\rightarrow A = 500, B = 400$
- ✓ If transaction T fails after T1 but before T2, the amount is deducted from A but not added to B. This shows an inconsistent database state.
- ✓ Therefore, the transaction must be executed completely or not executed at all to maintain correctness.

2. Consistency

- ✓ Consistency ensures that the database remains **correct before and after a transaction.**
- ✓ A transaction changes the database from **one consistent state to another consistent state.**

Example:

The total amount must be maintained before or after the transaction.

- ✓ Total before T occurs = $600+300=900$
- ✓ Total after T occurs = $500+400=900$

Therefore, the database is consistent.

In the case when T1 is completed but T2 fails, then database inconsistency will occur.

3. Isolation

- ✓ Ensures that one transaction's data is **not accessed by another transaction** until it completes.
- ✓ Transactions are executed independently.

Example:

If T1 is using data X, T2 cannot access X until T1 finishes

4. Durability

- ✓ Once a transaction is **committed**, changes are permanent.
- ✓ Data will not be lost even if system failure occurs.
- ✓ Ensured by recovery mechanisms (logs, backup)

Concurrency Control

Schedules:

A schedule is a **sequence of operations of one or more transactions.**

It is used to **maintain the order of operations within each transaction.**

Types of Schedules:

There are three types of schedules:

1. Serial Schedule

2. Non-Serial (Concurrent) Schedule

1. Serializable Schedule
2. Non-Serializable Schedule

1. **Serial Schedule:** A serial schedule is a schedule in which one transaction is executed completely before another transaction starts. There is no interleaving of operations between transactions.

For example: Suppose there are two transactions T1 and T2 which have some operations.

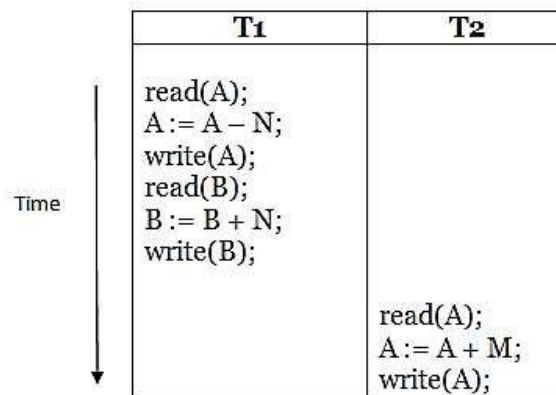
T1: read(A); A := A - N; write(A); read(B); B := B + N; write(B)

T2: read(A); A := A + M; write(A)

If it has no interleaving of operations, then there are the following two possible outcomes:

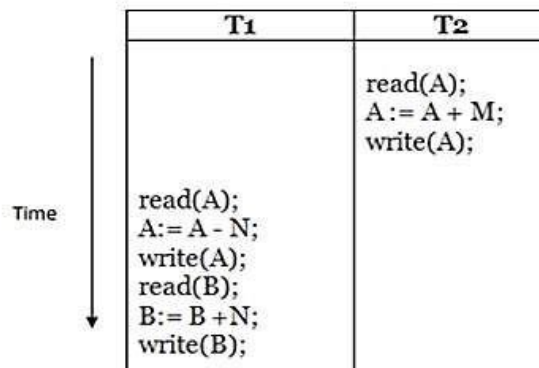
1. Execute all the operations of T1 which was followed by all the operations of T2.
2. Execute all the operations of T2 which was followed by all the operations of T1.

(a)



Schedule A

(b)



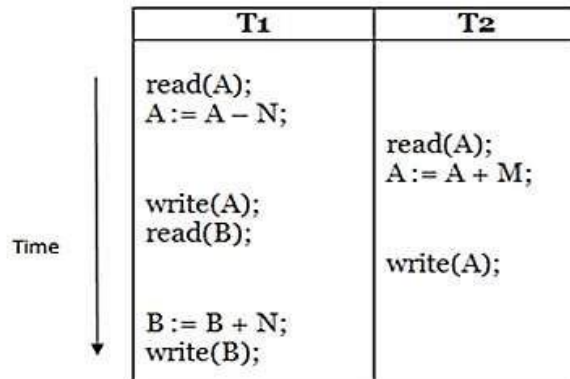
Schedule B

- In the given (a) figure, Schedule A shows the serial schedule where T1 followed by T2.
- In the given (b) figure, Schedule B shows the serial schedule where T2 followed by T1.

2. Non-serial Schedule:

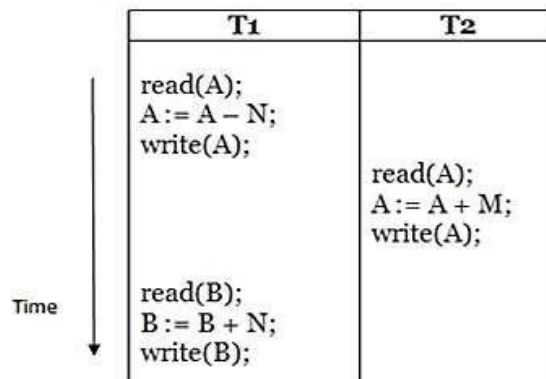
- If interleaving of operations is allowed, then there will be non-serial schedule.
- It contains many possible orders in which the system can execute the individual operations of the transactions.

(c)



Schedule C

(d)



Schedule D

In the given figure (c) and (d), Schedule C and Schedule D are the non-serial schedules. It has interleaving of operations.

3. Serializable schedule:

- The serializability of schedules is used to find non-serial schedules that allow the transaction to execute concurrently without interfering with one another.
- It identifies which schedules are correct when executions of the transaction have interleaving of their operations.
- A non-serial schedule will be serializable if its result is equal to the result of its transactions executed serially.